Architecture of the AI Assessment Sandbox Configurator

Version 0.3 – 23 March 2026

Table of Contents

Introduction **2**

Definitions and Requirements **2**

Definitions: AI Technical and Regulatory Sandboxes 2

Requirements for AI Regulatory Sandboxes 3

Lessons learned from assessing AI systems in Technical Sandboxes 4

Requirements for Self-Assessment and Conformity Certification 4

User Journey of Sandbox Configurator for Digital Sandboxing in AIRSes **5**

1. Aggregation and Harmonisation of Testing Solutions 5

2. Flexible and Portable Deployment 6

3. Role-Based Dashboards and Structured Collaboration 6

Proposed Architecture of the AI Assessment Sandbox Configurator **6**

Catalogue of AI Tests and Controls 7

Qualification of AI systems 7

Wizard to select tests & datasets 8

Configuration of collaborative assessment dashboard 8

Configuration of report generator 8

Configured AI Assessment Sandbox "à la carte" 9

Testing database 9

Test execution engine 10

Collaborative assessment dashboard 11

Report generator 12

Outputs 13

Tamper-proof audit logs 13

Exit Report(s) 13

Introduction

This document describes the requirements, user journey, overall architecture and components of the AI Assessment Sandbox Configurator and the resulting configured sandbox *à la carte,* i.e. a sandbox tailored for the requirements of a specific use case. Technical sections are highlighted with shaded background throughout the document.

Definitions and Requirements

As AI becomes increasingly powerful and pervasive across Europe's economy and society, the number of AI systems requiring formal assessment is growing rapidly. The EU AI Act establishes clear obligations for AI providers to demonstrate trustworthiness of their AI solutions and designates AI Regulatory Sandboxes as a key mechanism for structured dialogue between innovators and competent authorities.

Definitions: AI Technical and Regulatory Sandboxes

The term "sandbox" is overloaded with different connotations in software engineering and in the AI domain. Therefore, we propose the following definitions that will be used throughout this document. A key distinction exists between AI *Technical* Sandboxes (AITS) and AI *Regulatory* Sandboxes (AIRS). While both foster trustworthy AI, they are different in the sense that *Regulatory* sandboxes are supervised by National Competent Authorities under the framing provided by the AI Act (Articles 57 and 58).

In particular (cf. Figure 1).:

- **AI Technical Sandboxes (AITS)** are controlled, isolated environments that enable AI providers to safely develop, train, test, and validate their AI systems before market deployment, without real-world consequences. Typically, AITS include two environments: a *development* environment or sandbox with software tools, data, computational resources and experts required to create AI systems; and an *assessment* environment or sandbox to test that the AI solution is trustworthy and compliant. Usually, the creation of an AI system requires several iterations from development to testing and back to development until the system passes all relevant tests and controls required by the AI Provider. In some cases, both development and test environments are in the same infrastructure and are performed by the same team. In other cases, the AI provider may decide to separate to separate development and test on different infrastructure and/or different teams in the same organization or even to have the testing performed by a third party. For example, an AI provider may ask an independent Notified Body to perform a conformity certification of the AI system.
- **AI Regulatory Sandboxes (AIRS):** according to Article 3(55), an AI Regulatory Sandbox is defined as "a controlled framework set up by a competent authority which offers providers or prospective providers of AI systems the possibility to develop, train, validate and test, where appropriate in real-world conditions, an innovative AI system, pursuant to a sandbox plan for a limited time under regulatory supervision". The EUSAiR project, funded by the EC AI Office to develop AIRS guidelines for National

Competent Authorities and to conduct AIRS pilots across all 27 Member States ahead of the AI Act's official requirements, identified two types of AIRS[1]:

- **AIRS 1.0 focused on Regulatory Guidance** (cf. AI Act Articles 57.5, 57.6, 57.7, 57.11)
- **AIRS 2.0 also known as Digital Sandboxes** which combine regulatory guidance with technical support for developing, training, testing, and monitoring AI systems (cf. AI Act Article 58(2)(i))

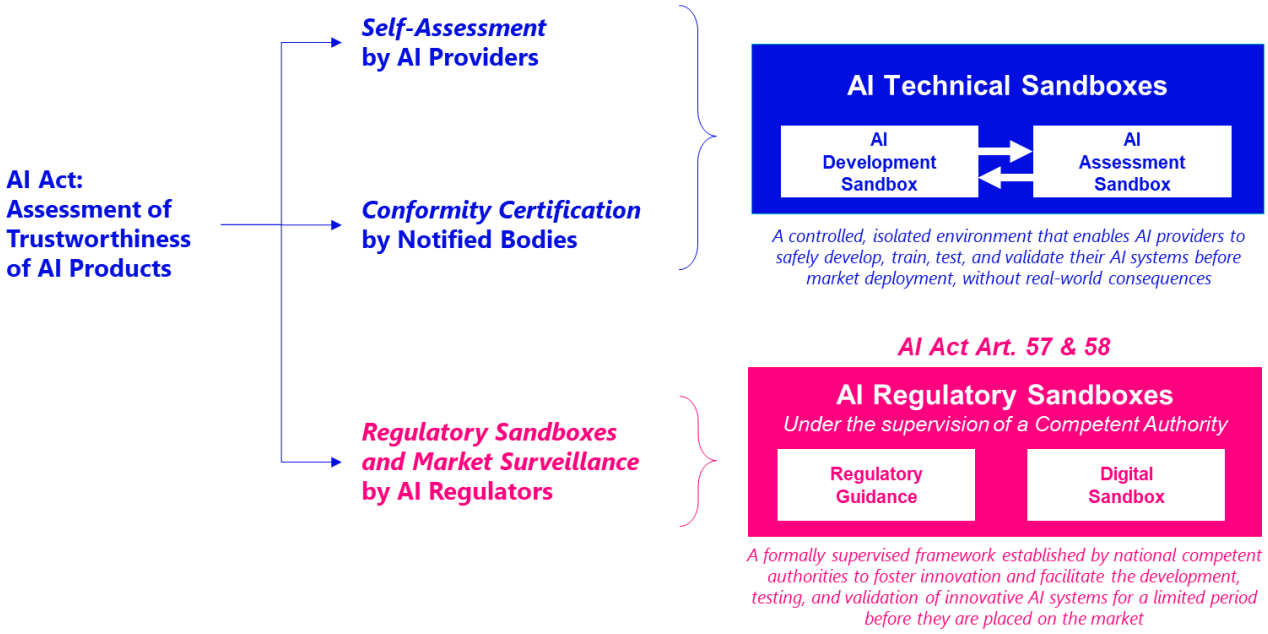

Figure 1 Definitions of AI Technical vs. Regulatory Sandboxes

Requirements for AI Regulatory Sandboxes

With National Competent Authorities (NCAs) required to have at least one AI Regulatory Sandbox operational by 2 August 2026, the detailed technical and operational requirements for Digital Sandboxes remain a work in progress across Member States. Several research and policy actors have been trying to anticipate the requirements and shape the tools and frameworks that NCAs will need before the deadline, for example:

- Emerging analysis by FARI highlights key outstanding questions around which authorities should operate sandboxes, what regulatory flexibility can legitimately be granted to participants, and how sandboxes might connect to technical testing infrastructures without creating conflicts of interest between regulators and service providers[2].
- Concrete implementation proposals have begun to emerge at the national level, informed by preliminary investigations and pilots: one such proposal advocates for a single multi-sectoral sandbox accessible via one entry point, in which all relevant supervisory authorities participate, so that AI providers do not have to determine independently which authority to contact, and regulators can jointly arrive at a consistent interpretation of the AI Act. It further recognises that while supervisory

[1] EUSAIR project: *The EU AI Ecosystem and AI Regulatory Sandboxes: Potential Synergies (preliminary assessment)*. July 2025. https://eusair-project.eu/library/

[2] N. Genicot: *From Blueprint to Reality: Implementing AI Regulatory Sandboxes under the AI Act*. December 2024. https://www.fari.brussels/research-and-innovation/publication/ai-regulatory-sandboxes

authorities should focus on legal and technical guidance rather than operating physical infrastructure themselves, existing instruments such as Testing and Experimentation Facilities (TEFs) and European Digital Innovation Hubs (EDIHs) can provide the technical testing support that AI Regulatory Sandboxes require, thus making the link between regulatory and technical infrastructure a key design consideration[3].

- Pilot simulations conducted at the national level have reinforced this picture, generating practical findings on cross-authority cooperation, maturity models for assessing AI systems under development, and the articulation between the AI Act and sector-specific regulation[4].

The architecture proposed in this document draws on two complementary foundations: on the one hand, the experience with technical sandboxes summarised in the next section; on the other, the analysis of literature on Regulatory Sandboxes and the dialogue with NCAs summarised in the paper *The Sandbox Configurator*[5]. Of course, this paper is not the definitive source of requirements and it is being continuously updated as inputs are collected from interactions and from the ongoing EUSAIR pilots.

Lessons learned from assessing AI systems in Technical Sandboxes

Our experience implementing AITSes in projects with partners ranging from startups and banks to a major GPAI provider has revealed three recurring lessons:

1. **Setup is complex and time-consuming**: Establishing a technical testing environment requires substantial effort to identify, select, and integrate appropriate tools and datasets, which are often heterogeneous and technically incompatible. Deployment constraints vary across organisations. In privacy-sensitive sectors, the entire assessment must be often executed on the partner's premises, further increasing setup complexity and time-to-operation (see Figure 2 for more details).
2. **One size does *not* fit all**: Each use case and vertical domain carries distinct risks, impacts, metrics, testing methods, and acceptance thresholds. As a result, a generic assessment approach is insufficient. Effective evaluation must be adaptable to the specific technological, regulatory, and operational context of each deployment.
3. **Multi-disciplinary collaboration is essential**: Testing and optimising AI systems requires coordinated input from diverse experts across departments and organisations, including technical, legal, compliance, business, and domain specialists. Without structured collaboration, assessment processes become fragmented and inefficient.

[3] Dutch Data Protection Authority: *Proposal Dutch regulatory sandbox*. 26 March 2025. https://www.autoriteitpersoonsgegevens.nl/en/documents/proposal-dutch-regulatory-sandbox

[4] Bundesnetzagentur: *Trilateral pilot project for AI regulatory sandbox simulation successfully concluded*. 17 March 2026. https://www.bundesnetzagentur.de/SharedDocs/Pressemitteilungen/EN/2026/20260317_Ki.html

[5] A. Buscemi, T. Simonetto, D. Pagani, G. Castignani, M. Cordy, J. Cabot: *The Sandbox Configurator: A Framework to Support Technical Assessment in AI Regulatory Sandboxes*. September 2025. arXiv:2509.25256

Bottlenecks in setting up a tailored AI technical sandbox:

1. Translating requirements into metrics and tests
2. Finding the relevant tests from disparate sources
3. Installing each test into the sandbox environment
4. Manual system integration to execute the tests
5. Manually harmonize the results of each test into a coherent database and dashboard
6. Manually collect evidences of tests performed
7. Interpreting the test results with feedback from different experts
8. Manually compiling the report

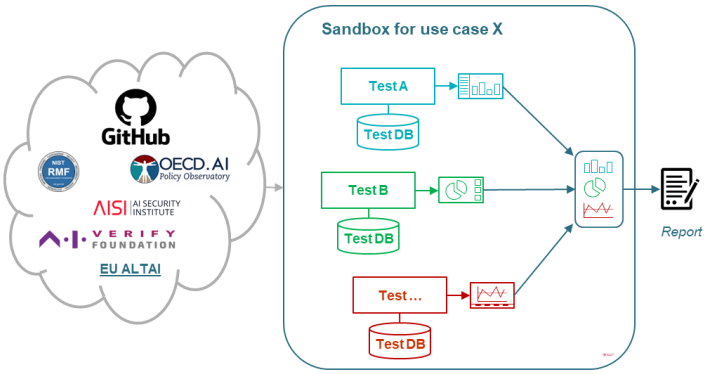

Figure 2 - Bottlenecks in setting up a tailored AI technical sandbox

Requirements for Self-Assessment and Conformity Certification

While the Sandbox Configurator has been designed primarily to support AIRSes, its architecture also enables its use for both **self-assessment** and, in principle, **conformity assessment**. AI providers and deployers can use it to perform structured self-assessments, select relevant tests from the Catalogue, and generate harmonised reports to support internal governance and compliance preparation. By enabling relatively easy iterative testing and consistent evaluation across development cycles, the Configurator also guides meaningful compliance-by-design practices, thus helping teams embed regulatory requirements into AI solutions from the outset rather than retrofitting them later.

Regarding conformity assessment, the full operational alignment with formal procedures is not yet implemented, as the relevant implementing acts and harmonised standards are still under development. We will closely follow these regulatory developments and adapt the Configurator accordingly. Nevertheless, the current architecture already provides the technical foundations—reproducible testing, traceability, and harmonised reporting—that can support future conformity assessment requirements once the regulatory framework is fully specified.

User Journey of Sandbox Configurator for Digital Sandboxing in AIRSes

To facilitate digital sandboxing within AIRSes, we have designed the AI Assessment Sandbox Configurator, an open architecture to reduce the time and effort required to define an AI assessment sandbox “à la carte”, tailored for a specific use case, sector and testing requirements. The configured sandbox is deployable in any environment supporting containerisation technologies (e.g. Kubernetes, OpenShift). We are currently implementing a first Proof of Concept of this architecture, which we plan to release at the end of May 2026 as open source (Apache 2 license). Subsequent releases are scheduled after the summer.

The user journey, illustrated in Figure 3, starts with the installation of the Sandbox Configurator on the infrastructure chosen by the user (on premises, AI Factory, cloud, etc.). Then the user configures the endpoints to connect to the AI system under assessment and fills out an *AI system qualification form.* This form captures

the AI system's risk category, the types of end users and third parties affected, and the specific aspects to be tested. The system then transforms this into an *AI System Card*, i.e. a machine-readable structured profile of the AI system under assessment.

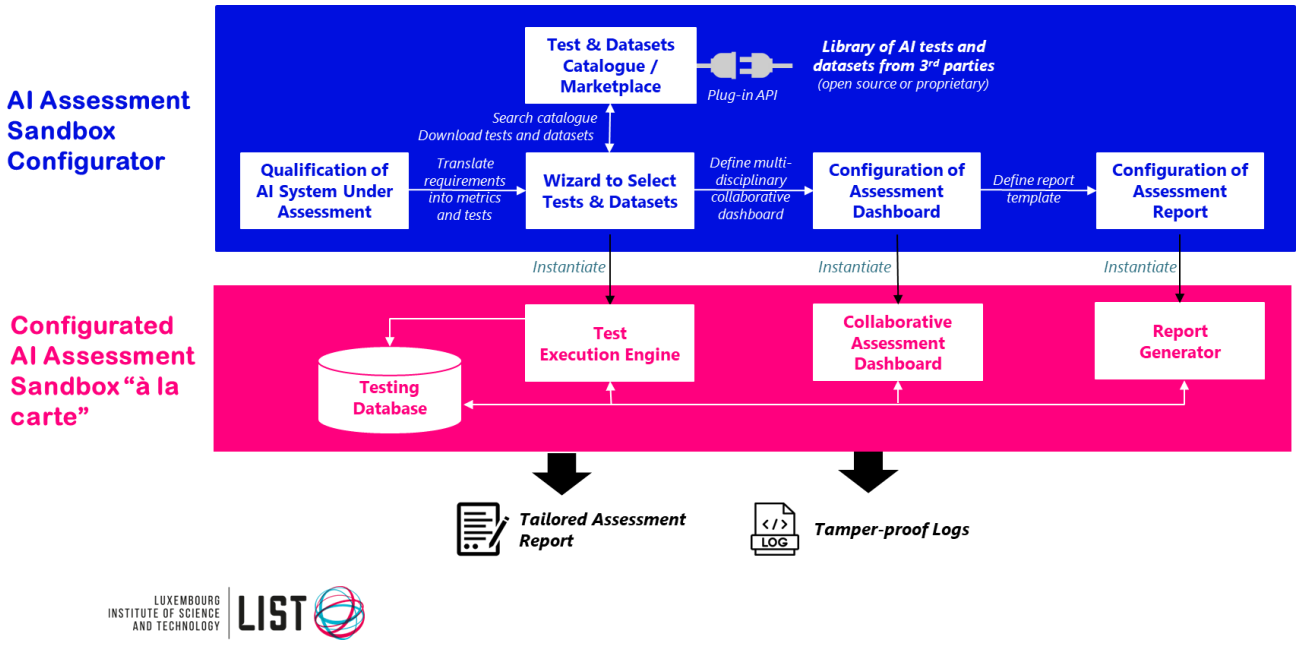

Figure 3 – User Journey of the AI Assessment Sandbox Configurator and the Configured Sandbox "à la carte"

Based on the information in the AI system card, a *Wizard* supports the user in translating requirements into metrics, tests and controls by selecting them from a *Catalogue of AI Tests and Controls.* This Catalogue, illustrated in Figure 4, aggregates many AI tests and controls that have been integrated into the system using a plugin API. Once the user has selected which AI tests and controls s/he wants to include into his/her tailored Sandbox, s/he then proceeds to configure the *Collaborative Assessment Dashboard,* choosing the types of visualization for each test results and the stakeholders / multi-disciplinary experts who will have access (e.g. business process owners, regulators, technical AI experts, legal experts, compliance and risk management experts, ethical experts, etc.). The last step of the configuration concerns the *Assessment Exit Report*: the user can select from a library of existing report templates or create a new report template with the desired formatting style, logos, and content.

Now we have reached a key moment of the user journey: the user has selected the test and controls that are relevant for the AI system and use case under assessment, has configured the collaborative dashboard and has chosen the report template(s), so s/he is ready to ask the Configurator to generate an AI assessment sandbox tailored for that specific use case. The generated sandbox can be deployed on any container infrastructure, such as Kubernetes or OpenShift on premises, on AI Factories or in the cloud.

The generated sandbox allows tests and controls to be executed via the UI or API through an execution engine. Test results are stored in a common *Testing Database* and all actions are recorded in *Tamper-proof Audit Logs*. The *Collaborative Assessment Dashboard* supports interpretation of results and collaborative annotations by the

multi-disciplinary panel of technical and non-technical people, facilitating communication between different types of stakeholders and experts involved in the assessment.

After one or more iterations of testing, the *Report Generator* produces a consistent and machine-readable *Exit Report,* based on the selected template and designed to share insights across developers, deployers, regulators, and affected communities, and to support regulatory learning from sandbox participation.

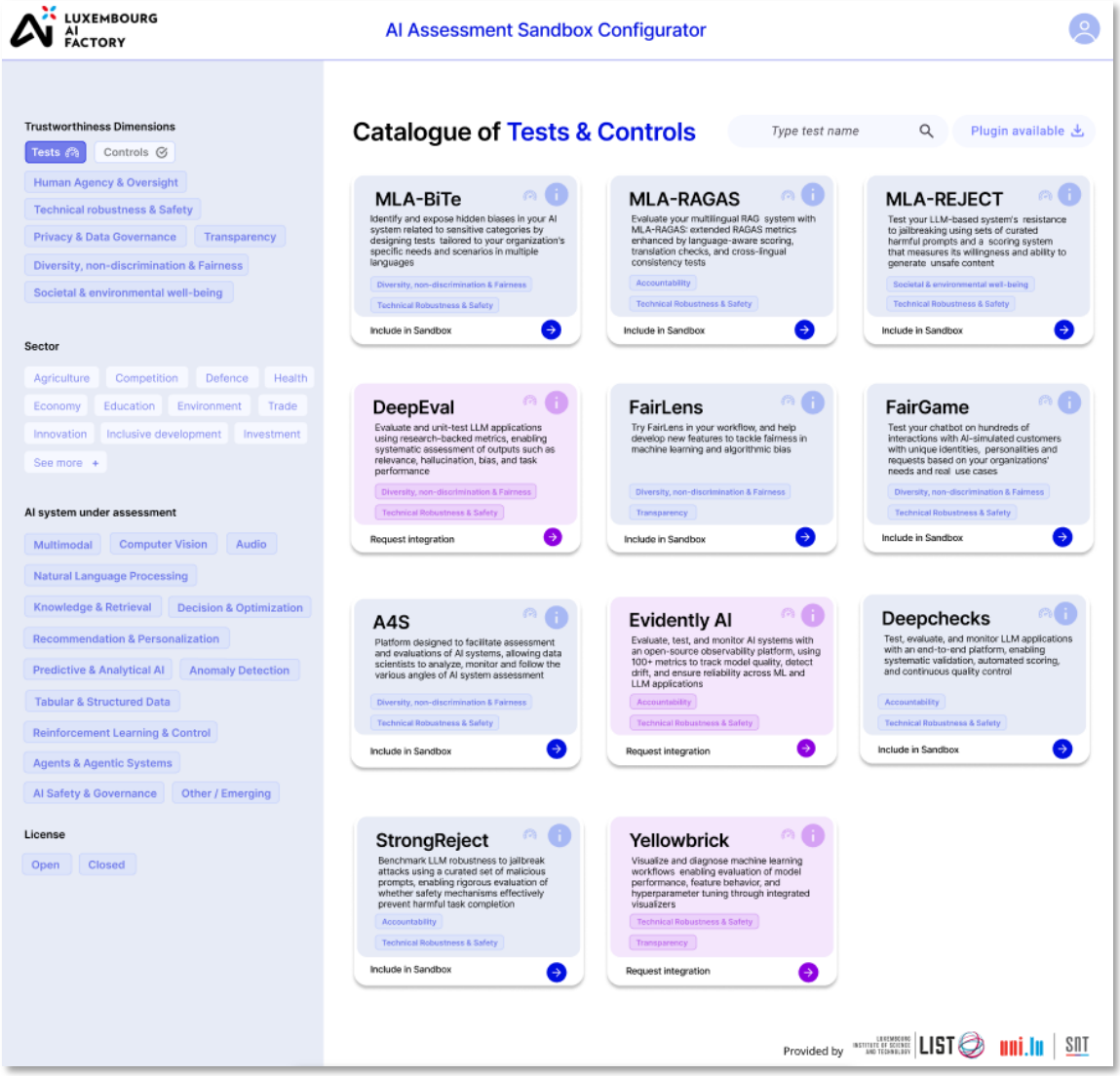

Figure 4 - User interface of the Catalogue of Tests and Controls

The AI Assessment Sandbox Configurator addresses the challenges outlined above through three key features described in the next sections.

Feature 1: Aggregation and Harmonisation of Testing Solutions

The Configurator aggregates a broad range of testing solutions and datasets from diverse sources into a unified environment referred to as the **Catalogue** (with potential future evolution into a marketplace). These testing solutions are developed independently by startups, corporations, research centres, and open-source communities. They:

- Target different AI technologies,

- Evaluate different dimensions (e.g., performance, robustness, bias, explainability),
- Produce outputs in heterogeneous formats.

Each testing solution is integrated into the Catalogue through a **plugin API**, i.e. a self-contained adapter that encapsulates the solution's execution logic and maps its outputs to the system's shared domain model. This plugin-based approach serves as the harmonisation layer, enabling:

- Execution of heterogeneous tests within a unified framework,
- Standardised visualisation of results,
- Harmonised reporting across tools and domains.

Feature 2: Flexible and Portable Deployment

Testing, visualisation, and reporting can be conducted by deploying the software in an infrastructure of choice, including:

- On-premise environments (e.g., privacy-sensitive organisations),
- AI Factory HPC infrastructures,
- Evaluators' premises,
- Cloud or hybrid infrastructures.

The Sandbox Configurator is packaged and orchestrated using Docker Compose, enabling modular containerised deployment. As a result, it can run on any infrastructure that supports containerisation technologies, including platforms such as Kubernetes and OpenShift.

A single deployment provides a fully operational testing and reporting environment, significantly reducing integration overhead while ensuring portability, scalability, and reproducibility across different infrastructures.

Feature 3: Role-Based Dashboards and Structured Collaboration

The Sandbox Configurator provides tailored dashboards for different stakeholder groups, ensuring that each user can focus on information relevant to their role and level of expertise. The platform enables stakeholders to:

- Comment on results,
- Provide structured feedback,
- Plan and coordinate testing activities,
- Track assessment progress.

By serving as a collaborative space, the Configurator supports multi-disciplinary coordination and reduces fragmentation in AI assessment processes.

Proposed Architecture of the AI Assessment Sandbox Configurator

The proposed architecture of the AI Assessment Sandbox Configurator is composed of building blocks illustrated in Figure 5. The core building blocks are the Catalogue of AI Test and Controls and the Testing Database: these

are building blocks that allow the discovery integration and integration of different tests into one consistent sandbox with one database. The other building blocks can be

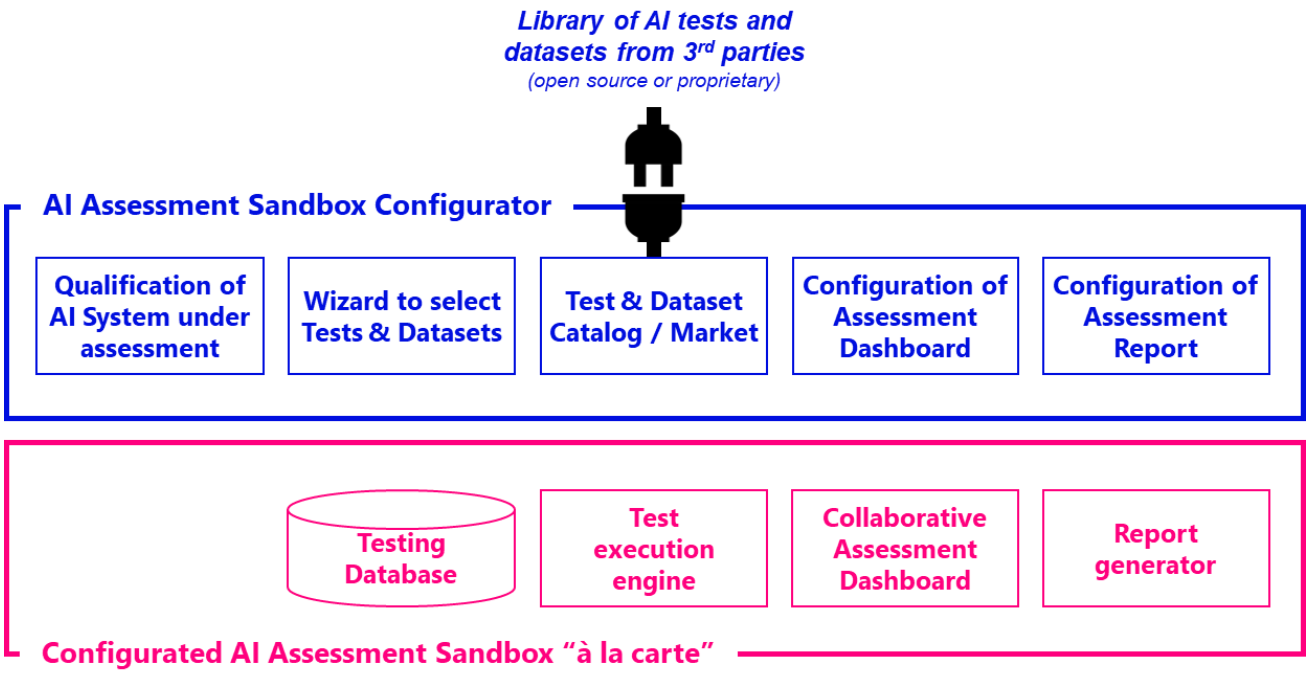

Figure 5 - Architecture of the AI Assessment Sandbox Configurator

Catalogue of AI Tests and Controls

The Catalogue is a library of testing tools and datasets from various sources (startups, research centres, open-source communities, etc.), each integrated via a plugin that wraps the tool's logic and maps its outputs to the shared domain model. This makes heterogeneous tools, evaluating different dimensions, producing different output formats, interoperable within a single framework. It may eventually evolve into a marketplace. For more details on the Catalogue and the Plug-in API, please consult the following documents on the Github repository https://github.com/lux-ai-factory/rfc: Catalogue of AI Tests and Controls and Plugin Developer Guide.

Qualification of AI Systems

This module is responsible for qualifying the AI system under assessment. It takes the form of a questionnaire to be completed by the organisation submitting the system for evaluation. The questionnaire must strike a balance between granularity, i.e. capturing enough detail to qualify the system accurately and unambiguously, and brevity, so as not to place an undue burden on participating organisations.

We have not implemented a qualification module, as we are currently scouting similar initiatives. To cite one, the questionnaire used for the AIRS plan developed by EUSAiR. This questionnaire and other initiatives, provide a comprehensive list of requirements to check. Whatever form we ultimately choose to adopt, we believe that multiple versions of the questionnaire should be available, tailored to different company sizes and sectors. A

large organisation in the healthcare sector, for instance, is likely comfortable submitting detailed forms, whereas a startup in the video games industry is not. This tiered approach also aligns naturally with the risk categories defined under the AI Act, whereby the depth of qualification required scales with the level of risk associated with the system.

Wizard to Select Tests & Datasets

Once the system is qualified, the output is distilled into an AI system card: a machine-readable, structured representation of the qualification form. The system card serves two purposes. First, it drives plugin selection, determining which plugins from the catalogue are required to assess the specific system under review. In the first release, this selection will be performed manually by the user; in a subsequent step, we plan to introduce a multi-agent system (MAS) that generates the system card from the form and then identifies the relevant plugins automatically.

This MAS will be designed with explicit control points to keep the decision-making partially explainable and auditable, while the final call always rests with the human user, who can accept the recommendation as-is or add and remove plugins as needed. Second, machine-readable system cards enable structured data mining in support of regulatory learning: if the Sandbox Configurator were widely adopted as a foundation for digital sandboxes in AIRSes, this standardisation could meaningfully facilitate the sharing and comparison of experiences across jurisdictions and deployments.

Configuration of Collaborative Assessment Dashboard

Once all plugins have been selected, the assessment dashboard is configured. At this stage, the user defines for each plugin which plot types should be displayed and which access roles should be assigned to the users interacting with the dashboard, so that results are surfaced in a way that is relevant to each audience.

> **Technical details**
>
> In our implementation, the BESSER Web Model Editor (WME) is used to build a collaborative dashboard within the AI Sandbox environment. The WME UI builder provides a no-code interface where users can compose dashboards by dragging and dropping components such as plots, charts, tables, and key metrics. Each component can be configured by selecting the relevant tables and columns from a shared database that integrates data from multiple assessments.

In addition to the built-in dashboarding capabilities, the underlying database is exposed via a FastAPI service. This allows advanced users to directly access the data and develop custom visualizations or user interfaces using tools of their choice, enabling both low-code and fully customizable workflows.

Configuration of Report Generator

Once all plugins have been selected and the dashboard has been configured, the report generator is configured. At this stage, the user defines for each plugin which plots and results should be included in the report and which sections should be visible to which audience, so that the report is tailored to the needs of each stakeholder group.

We are currently building a module in which users get a view based on the installed plugins and can then select their desired level of granularity for the optional information of the plugin-specific sections. Some information for each plugin will still be mandatory and therefore not subject to exclusion in the report. This way, a simplified version of the report can be used as a communication tool with non-technical stakeholders providing an

overview of the system assessment, whereas the in-depth version will include all technical details for experts to consult.

Configured AI Assessment Sandbox "à la carte"

Testing database

We have designed a **shared domain model** to enable consistent interpretation and comparison of results across heterogeneous testing solutions (see Table 3). As a foundation, we adopted the **Structured Metrics Metamodel (SMM)** defined by the Object Management Group (OMG)[6], represented in the table by the white and grey classes. Building on SMM, we extended the metamodel in two major iterations (illustrated by the green and blue classes) to accommodate concepts specific to AI system assessment, including AI system metadata, datasets, and testing configurations. These extensions ensure that results are not represented in isolation, but together with the contextual information necessary for traceability and reproducibility.

The model has been tested against a diverse set of plugins, ranging from traditional predictive performance evaluation (e.g., classification and anomaly detection) to testing frameworks for large language models. Despite significant differences in technologies, evaluation dimensions, and output formats (CSV files, structured JSON etc.), the schema has proven sufficiently robust to harmonise and represent heterogeneous outputs under a common structure. As new plugins and use cases are integrated, the model continues to be validated in practice. Where necessary, the schema can be extended in a controlled and systematic manner, ensuring both stability and adaptability over time.

As a simple example, consider a plugin such as *ACMETester* evaluating an anomaly detection system and producing a CSV file with metrics such as F1-score or Accuracy. In the shared model, the tool metadata (name, version, licence) is captured under the **Tool** entity, the mathematical definition of F1-score is represented as a **Derived Metric**, and the computed result (e.g., 0.89) is represented as a **Measure**, optionally including error or uncertainty if reported.

Tool	**name**	AcmeTester
	source	ACME Co.
	licensing	Apache 2.0
	version	2.0
Derived Metric	**expression**	[insert formula]
Measure	**value**	0.89
	error	0.03
	uncertainty	[0.85, 0.92]
	unit	Dimensionless

[6] OMG: *Structured Metrics Metamodel*. March 2018 - https://www.omg.org/spec/SMM/1.2/About-SMM

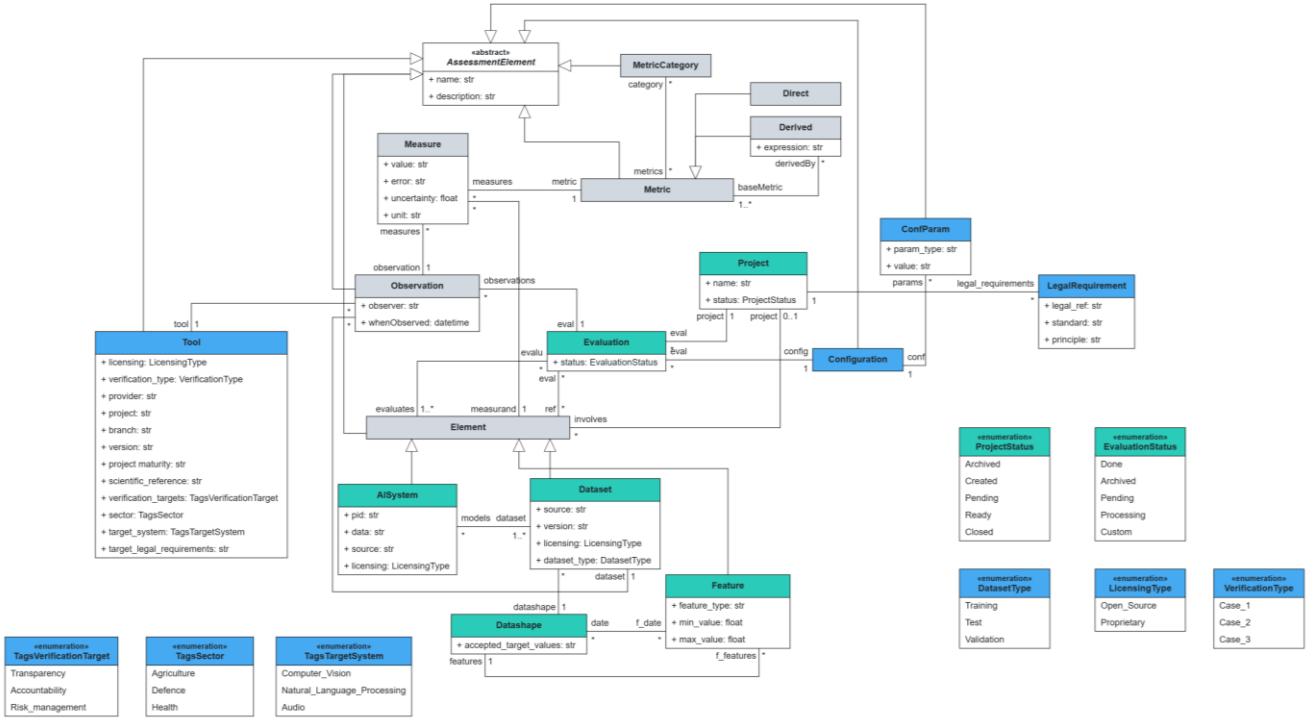

Figure 6 - Shared domain model for results harmonisation

Test execution engine

This module enables tests to be conducted in an automated manner and is characterised by the following capabilities:

- **Connectivity to the endpoints** of the target AI system under assessment.
- **Support for uploading test sets directly** or connecting to databases and persistent storage where test sets are held.
- **Versioned run configurations per plugin**, allowing tests to be executed with different parameters to reflect varying testing logic and ensuring reproducibility.
- **Role-Based Access Control (RBAC)**, enabling fine-grained permission management across different stakeholder groups, ensuring that users can only access, configure, execute, or review tests according to their assigned roles.
- **A dashboard for real-time progress tracking**, providing visibility into execution status and intermediate results.
- **Tamper-proof audit logs**, ensuring traceability of actions, configuration changes, and test executions for accountability and compliance purposes.
- **Extensible plugin orchestration mechanisms**, allowing new testing tools to be integrated and managed within a unified execution framework.
- **Connection to the database implementing the shared domain model**, where all results generated by the testing processes are stored in a structured and harmonised manner.

Technical details

In our implementation, we use a distributed architecture designed for scalability and extensibility, comprising the following elements:

Web UI (Frontend): The web interface is built with React and TypeScript, utilizing Vite as the build tool. The UI components are primarily based on Material UI (MUI), with React Router for navigation and TanStack Query (React Query) for efficient data fetching and synchronization with the backend. Data

visualization is handled by Chart.js and MUI X Charts, while complex dynamic forms for plugin configuration are generated using react-jsonschema-form (RJSF).

Backend Application: The backend is developed using Python 3.12 and the Django framework. It leverages Django Ninja for building high-performance, type-annotated RESTful. Data persistence is managed through PostgreSQL, and the application is designed to be containerized and deployed with Docker.

Evaluation Runner: The evaluation runner is a separate service that handles heavy computational tasks asynchronously. It uses Celery as the task queue, with RabbitMQ as the message broker and Redis as the result backend.

Extensibility: The runner uses the plugin-based architecture described in this document, where individual evaluation logic is encapsulated in plugins that adhere to a common interface.

- **Monitoring**: Task execution and worker health can be monitored in the Web UI.
- **Communication & Integration**: The backend communicates with the evaluation runner by dispatching Celery tasks. The runner, in turn, interacts back with the backend's API to report progress, post evaluation measures, and update the status of evaluation runs.

Logging: Immutable audit logging is handled by ImmuDB, an open-source append-only database that enforces tamper-evidence cryptographically via a Merkle hash tree. Records cannot be modified or deleted, even by an administrator. ImmuDB is accessed via its official Python SDK through a gRPC API and runs as a standalone container alongside the rest of the system.

It is important to note that this implementation of the test execution engine is the default solution we provide. However, alternative solutions can in principle be used (e.g. MLFlow), provided they are made compatible with the Sandbox Configurator, that is they consume the plugins and write results to a database adopting our domain model.

Collaborative assessment dashboard

The dashboard is characterised by the following features:

- **Connection to the database implementing the shared domain model**, enabling the dashboard to consume all results.
- **Role-Based Access Control (RBAC)** to manage permissions and ensure that stakeholders access only the functionalities and results relevant to their roles.
- **Aggregation of outcomes** across multiple plugins and test runs, providing a consolidated and harmonised view of results.
- **Interactive visualisations**, including filtering capabilities to focus on specific subsets of results (e.g., by dataset, configuration, metric, or test run).
- **Commenting functionalities**, enabling stakeholders to provide feedback, document observations, and support coordinated assessment activities.
- **Integrated report generation**, allowing users to produce structured reports directly from the dashboard.

Technical details

In our implementation, the webapp is generated deterministically using BESSER, a model-driven low-code/no-code framework that automatically produces application code from high-level models. BESSER is used to generate React-based frontend and a Python backend, ensuring consistency between system design and its implementation. The frontend is implemented as a React application written in TypeScript and provides user interface (e.g. charts, tables, pages) used to visualize and interact with the data. It communicates with the backend through API calls to retrieve and display information.

The backend is implemented in Python using FastAPI, which exposes the REST API endpoints for communication with the frontend. Data validation and schema definition are handled using Pydantic, while SQLAlchemy is used as the ORM layer to manage interactions with the database. To ensure portability and reproducibility, both frontend and backend service are containerized using Docker, enabling consistent deployment across environment.

It is important to note that this implementation of the dashboard is the default solution we provide. However, alternative business intelligence or visualisation platforms (e.g., Power BI, Tableau) can in principle be used, provided they are made compatible with the Sandbox Configurator, that is they can access and interpret results stored in a database implementing the shared domain model.

Report generator

This module is responsible for generating reports tailored to the plugins used in the evaluation. It can be triggered directly from the assessment dashboard or executed independently as a standalone module. The report generator is characterised by the following features:

- **Connection to the database implementing the shared domain model**, enabling the report generator to consume all results.
- **Fully automated**. Once the report configuration for different stakeholder groups has been done, the generator automatically generates the PDF report according to the user-provided configurations.
- **Template-based and extensible**. The orchestration of the report generator is decoupled from the plugins, allowing for easy extension via new templates.

Technical details

In our implementation, we use Python as the orchestration layer leveraging a technology-agnostic SQLAlchemy metamodel for database access. Each plugin defines its own HTML-based Jinja template and the logic required to retrieve the relevant data from the database. This data is then injected into the template before being passed to the main module, which defines the overall structure of the report. Once the Jinja engine has rendered the HTML output in an intermediate step, the PDF report is generated using the Weasyprint library.

It is important to note that this implementation of the report generator is the default solution we provide. However, alternative solutions can in principle be used, provided they are made compatible with the Sandbox Configurator, that is, they can read results from a database adopting our domain model.

Outputs

Tamper-proof audit logs

All logs produced by the execution engine and the dashboards are exportable. In the current implementation, export is supported in CSV and JSON formats, with the architecture allowing for additional formats to be accommodated in future releases, such as:

- PDF with formatted tables and charts for formal compliance reporting
- SIEM-compatible formats (CEF, LEEF) for integration with security platforms like Splunk or QRadar
- Parquet/Arrow for large-scale analytics in data science pipelines
- OpenTelemetry (OTLP) for feeding audit events into observability stacks (Grafana, Datadog)

Exit Report(s)

The exit report consolidates the following:

- The system card as generated by the wizard.
- A summary of all tests performed, including global timestamps, the list of used plugins, who conducted the tests and under what conditions.
- For each plugin, the assessment dashboard plots with their associated stakeholder comments, alongside a technical deep-dive for expert review and auditing purposes.

The system may generate more than one report based on different templates (e.g. confidential vs. public exit reports).